

COS10004 Assignment 2 Report

Student ID: SWS00812

Student name: Vu Thanh Nam

The purpose of this report paper:

In this article, the architecture and operation of the stopwatch program are thoroughly examined. To understand the program's coding methods and their uses, we investigate its technical details. Readers will have a deeper understanding of the stopwatch's capabilities by the end.

How the program works:

The program initializes the screen with the terms "Stopwatch," "Elapsed Time," and "Split Time," and sets up interrupts for keypad input. Subsequently, it goes into an endless cycle, requesting keyboard input. It performs several operations upon keypress, all of which is a timer that increments the time values shown on the screen when we press "C", press "P" to pause the timer, press "R" to reset the stopwatch, and finally the keyboard button "S" for splitting functionality with 5 split-time displays. The code uses a delay function for one-second intervals and subroutines for various time-related tasks. The main loop manages related operations while continuously monitoring the keypad for input. when the program reaches "HALT" instruction, it ends.

Design and Solution Overview:

After starting the program, it will run through the set of instructions used to display the word "Stopwatch", "Elapsed Time", and "Split Time" out on the screen in a nice position, and the program will be stuck in an "infinite loop" with the Branch instruction to the **"keypress"** interrupt routine to check the interrupt routine every time the keyboard is pressed. After pressing the "C" button It will run through the "stopwatch" function. Inside the function, we will first see a Branch instruction to the **"increment_second_ones_row"** function which continuously increments the second-ones-row time to "9" and resets to "0". It's important to assign the register LR with the address 0x5c4 because it will return to line 439 after meeting RET instruction in the **"increment_second_ones_row"** function and continue the stopwatch. I added BEQ instruction to the **"increment_second_tens_row"** function, when the second one-row time is "0", which means it reset, it will increment the second tens row 1 time. After that, I used BEQ instruction to the "Reset_seconds" function to reset the seconds' rows to "0" after the second tens row time of "9". When the second tens row time is "0", which means it reset, the next BEQ instruction to the **"increment_minute_ones_row"** function is to increment the minute ones row 1 time. Moreover, when the R3 equals value 58, the program will branch to the **"increment_minute_tens_row"** and **"Reset_minute_ones_rows"** functions, and it will increment the minute tens row one time before resetting minute one's row to "0". After the time reaches "99:99", it will reset to "00:00" by the function **"Reset"**. Furthermore, the Branch instruction to **"keypress2"**

interrupt routine to check the interrupt routine every time the keyboard is pressed. Finally, It will reach the last Branch instruction to loop back the “stopwatch” label.

User Interaction:

- Press “C”, to start the program.
- Press “P”, to pause the program.
- Press “S”, to split out the current time of the stopwatch.
- Press “R”, to reset the whole stopwatch.
- Press “E”, to end the program.

Program Execution:

“keypress” interrupt

Inside the keypress interrupt routine, I load the value of the “.LastKey” instruction into register R0 by LDR instruction. Then, I compare the value in R0 with the ASCII code for 'C' which is 67. After that, the program will branch to the stopwatch label if the condition is true by BEQ instruction. Finally, it automatically branches to the “infinitemloop” label. It returns from exception by RFE instruction.

“keypress2” interrupt routine:

Load the value of the “LastKey” instruction into register R0 using LDR. it then compares the value in R0 with ASCII codes for specific keys ('P', 'E', 'R', 'S'). and Branch to corresponding labels (pause, End, Reset, splitorder) based on the comparison results.

For the 'S' key, load the value of “LastKeyAndReset” and branch to the “splitorder” label if the condition is true. When we press ‘S’ 1 time it will branch to the “split1” function, press it twice it will branch to the “split2” function, and so on until “split5”

This routine facilitates keypress handling, directing program flow based on the pressed key's ASCII code.

“Increment_second_ones_row” function

The function “increment_second_ones_row” is responsible for controlling the display of seconds on a digital interface. It starts by adding 1 to register R1. To maintain and restore the value of R2, a register that is essential for interacting with the character

screen, stack operations are used. The function stores the incremented value at the proper position, updating the display.

After that, it uses a function to guarantee a regulated delay and handles the Link Register (LR) with care. After that, the function branches to "keypress2," which makes it easier to carry out the remaining steps. With its clever boundary condition management, which resets the second digit if it goes above '9,' it guarantees smooth operation.

"increment_second_tens_row" function

The second tens-digit display is controlled by the "increment_second_tens_row" function. To maintain its value—which is required for interacting with the character screen—it pops and pushes R3. It determines the memory address and modifies the display by increasing R2, which stands for the second tens digit.

After that, the function uses a subroutine to provide a regulated delay and takes extra care with the Link Register (LR). The lack of a branching instruction indicates that the flow is continuous and ends with the function making a simple return. The second tens digit in the digital interface is updated and displayed thanks in large part to this function.

"increment_minute_ones_row" function

The minute ones digit on the digital interface is updated and shown thanks in large part to the "increment_minute_ones_row" function. It determines the memory address using the base address of the character screen in R4 and increases the value in R3, which represents the minute one digit. This computed memory address is then used to store the modified value.

By adding and removing R4 from the stack, the method makes sure that it is preserved. Its linear structure demonstrates its simplicity, and the simple return from the subroutine at the end marks its conclusion. All things considered, this feature makes a substantial contribution to the digital display's precise depiction of the minute one digit and is essential to keeping it in sync with real-time.

"Increment_minute_tens_row" function

To update the minute tens digit on the digital display, the "increment_minute_tens_row" function is essential. By first pushing registers R5 and R4 onto the stack and then popping them off, it successfully maintains the preservation of those registers. The updated minute one's digit is stored at this location by the function, which uses R4 to compute the memory address for the minute tens digit. R4 is subsequently increased for

the tens digit. Similarly, the modified value is stored at the memory address for the minute tens digit, which is determined by R5.

This subroutine's simple linear structure and meticulous register handling make it crucial for preserving the displayed time's correctness. The minute tens digit's ability to be synchronized in real-time is one of its functions that enhances the digital stopwatch.

Delay function

This little bit of assembly code provides a delay subroutine for a predetermined amount of time. To maintain the values of registers R0, R1, and R2, it first pushes them into the stack. The delay time is then adjusted to one second (modifiable). R1 is loaded with the current time as the initial value. After the code enters a loop, it keeps track of the amount of time that has passed by deducting the beginning time (R1) from the current time (R2). It keeps looping until the elapsed time is larger than or equal to the required delay by comparing this time with the desired delay time (R0). Ultimately, it ends the delay by returning from the subroutine and restoring the saved register contents.

All reset functions

Sequentially updating the necessary memory locations to the ASCII value '0', the "Reset_seconds" procedure effectively resets the seconds digit. To save time, the process's repetitious elements are condensed. The minute ones and tens digits are also reset via the "Reset_minute_ones_rows" and "Reset" subroutines, respectively, in a comparable manner. Systematic digit resets are ensured by the strategic memory address changes and the usage of registers for computations and preservation. Branching to "endreset" and returning from the function, the subroutine ends. All in all, the digital stopwatch's precise timekeeping capabilities are enhanced by these succinct and organized reset features.

All split functions

The order of split times in a digital stopwatch can be adjusted with the "resetsplitorder" function. The split order is tracked using the register R10, which is increased by 1 for each iteration. The second ones, second tens, minute ones, and minute tens rows on the character screen are updated in the respective memory locations via a sequence of conditional branches (split1 to split5) that handle various split orders.

By adjusting the display positions by the split order, this function guarantees effective split-time organization. To preserve program integrity, the repetitious structure is streamlined and registers (R5-R8) are carefully preserved and restored. The method can easily adjust to various split orders thanks to the usage of conditional branching.

The code contributes to the general functionality of the digital stopwatch by providing an example of a methodical way to handle split times clearly and concisely. The company accomplishes the intended split-order alterations quickly while guaranteeing readability and maintainability.

Limitations and Future Improvements:

- The program is decorated beautifully enough. It will be redecorated after this assignment
- Have not reached stage 7. I will update its functionality with arrays to keep and hold value to display the split times respectively.